PASO 01 · INTEGRANTE 3

Construir la pantalla del proyecto

Guía del integrante 3. HTML, CSS y JavaScript para la interfaz. Revisión: 12 de octubre de 2026. Three.js se propone únicamente al final, después de integrar y comprobar todo.

Qué ya está listo

Parte	Punto de partida
Motor Python	Genera trabajos y lotes de hasta 3. Calcula tiempos y resultados. Implementa I/E/P/C.
API FastAPI	Crea simulaciones, consulta por ID y aplica acciones. El servidor lleva el reloj y conserva estado en memoria.
main	La API está integrada en el commit 808c3a4. Se verificaron 56 pruebas aprobadas.
Frontend actual	frontend/index.html y app.js solo muestran una página básica y API: ok. Tu tarea es construir la pantalla completa.

Tu trabajo principal

Crear una pantalla donde se ingrese la cantidad de trabajos, se inicie una simulación y se vea el estado completo. Conectar las teclas I/E/P/C y botones equivalentes. Mostrar errores de forma comprensible y recuperar la simulación al recargar la misma pestaña.

```
Usuario -> HTML/JavaScript -> API FastAPI -> motor Python
Usuario <- pantalla       <- JSON        <- snapshot
```

El servidor es la autoridad del tiempo. La pantalla consulta y representa lo que recibe. No incrementes reloj, ejecutado o restante por tu cuenta; tampoco generes trabajos ni calcules sus operaciones en JavaScript.

Orden de entrega

1. Preparar tu rama y ejecutar la base.
2. Construir HTML/CSS y conectar las tres rutas.
3. Implementar consulta periódica, controles y recuperación.
4. Probar con los integrantes 1 y 2, abrir PR e integrar.
5. Solo después, agregar Three.js como mejora opcional.

README.md y la introducción de docs/contrato-api.md aún dicen que la API está pendiente. El código actual ya la incluye. Usa sus rutas y el contrato de campos como referencia; actualiza esa descripción en tu entrega.

PASO 02 · INTEGRANTE 3

Preparar el equipo y tu rama

Trabaja en PowerShell desde tu propia copia. Necesitas Git, GitHub CLI (gh), Python 3.12 y un editor como VS Code. Para la interfaz inicial no necesitas React, Node.js ni una base de datos.

1. Iniciar sesión y descargar la copia

```
git --version
gh --version
py -3.12 --version
gh auth login --hostname github.com --git-protocol https --web
gh auth setup-git
gh auth status
```

Si falta una herramienta, instala Git, GitHub CLI o Python 3.12. Reabre PowerShell. En el login, copia el código al navegador y entra con tu propia cuenta. Debes aceptar la invitación al repositorio como colaborador.

```
git config --global user.name "TU NOMBRE"
git config --global user.email "TU CORREO DE GITHUB"
New-Item -ItemType Directory -Force "$HOME/Documents/Proyectos"
Set-Location "$HOME/Documents/Proyectos"
git clone https://github.com/iLaurx/sso-multiprogramacion.git
Set-Location sso-multiprogramacion
```

Reemplaza nombre y correo. Si ya clonaste el proyecto, entra a esa copia y omite clone. No copies la .venv de otro integrante.

2. Crear tu rama desde main actualizado

```
git status
git switch main
git pull --ff-only origin main
git switch -c feature/i3-interfaz
git branch --show-current
```

Antes de cambiar de rama, guarda cualquier trabajo pendiente. Si tu rama ya existe, usa git switch feature/i3-interfaz. La última línea debe mostrar ese nombre.

3. Instalar y ejecutar desde la raíz

```
py -3.12 -m venv .venv
.\.venv\Scripts\python.exe -m pip install -r requirements.txt
.\.venv\Scripts\python.exe -m pytest -q
.\.venv\Scripts\python.exe -m uvicorn backend.main:app --reload
```

Crea .venv solo si todavía no tienes una de Python 3.12. Abre http://127.0.0.1:8000/ para la pantalla y /docs para probar la API. Mantén la terminal abierta; Ctrl+C detiene el servidor. No abras index.html con doble clic ni uses otro servidor para la entrega básica.

PASO 03 · INTEGRANTE 3

Archivos y diseño de la pantalla

Empieza con una interfaz clara de dos dimensiones. Primero debe funcionar completa. Este esquema es una propuesta de distribución, no una captura de una interfaz ya implementada.

Simulación por lotes

Reloj · Estado · Lote · Pendientes

Cantidad [4] [Iniciar] I / E / P / C

En espera

ID · TME · Ejecutado · Restante

Proceso en ejecución

Operación · Tiempos · Barra de avance

Terminados

ID · Lote · Operación · Resultado / ERROR

Archivo	Qué hacer
Modificar frontend/index.html	Formulario de cantidad, botones, encabezado, panel activo, tablas y zona de mensajes.
Modificar frontend/app.js	Peticiones, ID guardado, consulta periódica, renderizado, teclas y estados de los botones.
Crear frontend/styles.css	Distribución, colores, tablas, tamaños de pantalla y estados visuales.
Crear docs/pruebas-frontend.md	Registrar los casos comprobados y sus resultados, con evidencia si el equipo la pide.
Actualizar README.md	Explicar uso de la interfaz, teclas, ejecución y limitaciones actuales.

Conectar los archivos sin complicar la base

```
<!-- Dentro de head -->
<link rel="stylesheet" href="/styles.css">

<!-- Al final de body; conserva este arranque -->
<script src="/app.js"></script>
```

Conserva la comprobación /api/health en app.js. Las pruebas actuales esperan ese script y esa ruta. Puedes usar #estado para la conexión y otro elemento para el estado de la simulación. Los cambios de estructura deliberados deben actualizar sus pruebas, sin quitar la comprobación.

Usa label para el campo de cantidad, input type="number" con min="1" y step="1", botones con texto y una zona de avisos con aria-live="polite". En móvil apila los paneles y permite desplazamiento horizontal de tablas. No dependas únicamente de colores.

PASO 04 · INTEGRANTE 3

Conectar las tres rutas

La página se sirve desde FastAPI. Usa direcciones relativas: /api/… Así navegador y API comparten el mismo origen y no necesitas configurar CORS para esta entrega.

Solicitud	Entrada y salida
POST /api/simulations	Enviar {"numero_trabajos":4}. seed es opcional y se usa para pruebas. Recibir 201 con ID y estado inicial.
GET /api/simulations/{id}	Sin cuerpo. Recibir 200 con el snapshot completo.
POST /api/simulations/{id}/actions	Enviar {"accion":"pause"}. Valores: interrupt, error, pause, continue. Recibir 200 con estado actualizado.

Un ayudante para leer JSON y detectar errores

```
async function pedirJSON(ruta, opciones = {}) {
  const respuesta = await fetch(ruta, opciones);
  const datos = await respuesta.json();

  if (!respuesta.ok) {
    const detalle = Array.isArray(datos.detail)
      ? datos.detail.map(e => e.msg).join("; ")
      : datos.detail;
    const error = new Error(detalle || "No se pudo completar");
    error.status = respuesta.status;
    throw error;
  }
  return datos;
}
```

fetch no lanza un error automáticamente por un 404 o 409: revisa respuesta.ok. El código anterior conserva status para decidir qué hacer. El llamador debe capturar también errores de conexión o de lectura del JSON. Referencia de fetch.

Ejemplo de creación dentro de tu función de inicio

```
const datos = await pedirJSON("/api/simulations", {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify({ numero_trabajos: cantidad })
});
simulacionId = datos.simulacion_id;
sessionStorage.setItem("sso.simulacionId", simulacionId);
renderizar(datos);
```

Antes valida Number.isSafeInteger(cantidad) y cantidad > 0. Convierte el input a Number: enviar "4" como texto da 422. Bloquea Iniciar mientras se crea para evitar duplicados y vuelve a habilitarlo si falla. La API conserva la validación final.

PASO 05 · INTEGRANTE 3

Consultar sin cruzar respuestas

La pantalla debe consultar aproximadamente cada segundo. También debe dibujar inmediatamente la respuesta de cada acción. Organiza todas las solicitudes de la simulación en una sola cola.

1. Una cola para crear, consultar y actuar

```
let cola = Promise.resolve();
function encolar(operacion) {
  const resultado = cola.then(operacion);
  cola = resultado.catch(() => {});
  return resultado;
}
```

Usa encolar para cada petición de creación, GET y acción. La operación encolada debe incluir leer el ID actual, pedir JSON y renderizar su respuesta. Así ninguna respuesta antigua sobrescribe una acción más reciente. Captura el error de resultado y muestra el aviso; la cola debe seguir disponible.

2. Un solo ciclo de consultas

```
// Esquema; implementar dentro del controlador de app.js:
await encolar(async () => {
  if (!simulacionId) return;
  const datos = await pedirJSON(
    `/api/simulations/${simulacionId}`
  );
  renderizar(datos);
});
// Tras terminar: programar la próxima consulta con setTimeout.
// Si completed o no hay ID: detener el ciclo.
```

Programa la próxima consulta con setTimeout después de terminar la anterior, incluyendo su catch/finally. Mantén un identificador de temporizador y una marca de ciclo activo. Al reiniciar el ciclo, cancelar el anterior. Evita un setInterval que acumule solicitudes si la API responde lento.

3. Reglas del controlador

Durante running: consultar y permitir I/E/P/C según su estado.
Durante paused: seguir consultando; el servidor devuelve tiempos iguales. Deshabilitar I/E.
Durante completed: conservar la última pantalla y detener consultas. Permitir iniciar una nueva simulación.

Al crear otra, detén el ciclo anterior y actualiza el ID dentro de la cola. Lo más sencillo para esta entrega es permitir una nueva creación cuando no existe simulación o cuando la anterior terminó. Evita abrir una nueva en cada recarga.

En una caída temporal, conserva el último snapshot y muestra “Sin conexión; datos sin actualizar”. Reintenta con una espera mayor, por ejemplo 3 segundos. No muevas tiempos localmente para aparentar que la API sigue conectada.

PASO 06 · INTEGRANTE 3

Mostrar el snapshot completo

Escribe una función renderizar(snapshot). Úsala después de crear, consultar y aplicar acciones. El motor ya entrega todos los datos: el frontend solo los acomoda y los presenta.

Zona	Campos que debe representar
Encabezado	estado, reloj, lote_actual y lotes_pendientes. En completed, mostrar “Finalizada” y lote “-”.
Espera	en_espera: ID, lote, TME, ejecutado y restante. Respetar el orden recibido; solo contiene el lote actual.
En ejecución	en_ejecucion: ID, lote, operación, operandos, TME, ejecutado y restante. Si es null, mostrar “Sin proceso activo”.
Terminados	terminados: ID, lote, operación, operandos, TME, ejecutado y resultado. Conservar el orden de terminación.
Resultado	Si estado del proceso es error, mostrar ERROR. Si finished, mostrar resultado numérico; para división, hasta 4 decimales.

Barra de avance y resultado

```
const porcentaje = proceso
  ? Math.min(100, 100 * proceso.ejecutado / proceso.tme)
  : 0;

function mostrarResultado(proceso) {
  if (proceso.estado === "error") return "ERROR";
  if (proceso.resultado === null) return "Pendiente";
  return Number.isInteger(proceso.resultado)
    ? String(proceso.resultado)
    : proceso.resultado.toFixed(4);
}
```

El formato de cuatro decimales es presentación. No modifiques ni sobrescribas el resultado guardado. Un terminado por error tiene restante 0 aunque no haya completado su TME. En ejecución y espera muestra los tiempos recibidos, sin recalcular el reloj.

Actualizar sin romper la pantalla

Crea los nodos con document.createElement y llena los valores con textContent. Reemplaza el contenido de los cuerpos de las tablas, por ejemplo con replaceChildren. No reconstruyas todo body: perderías listeners, foco del formulario y el futuro canvas.

Si las listas están vacías, muestra una fila “Sin trabajos en espera” o “Sin terminados”. No accedas a en_ejecucion.id sin revisar antes que existe. Representa paused con un aviso visible y completed con un resumen final.

Un indicador de “API disponible” puede seguir separado del estado de la simulación. El reloj visible siempre sale de snapshot.reloj, nunca de un contador del navegador.

PASO 07 · INTEGRANTE 3

Teclas, mensajes y recuperación

Botones y teclado deben llamar a la misma función de envío de acciones. Conservar el ID permite recuperar la simulación al recargar la misma pestaña, mientras el servidor siga funcionando.

1. Mapear las teclas

```
const acciones = {
  i: "interrupt", e: "error", p: "pause", c: "continue"
};

document.addEventListener("keydown", evento => {
  if (evento.repeat || evento.ctrlKey || evento.altKey ||
      evento.metaKey) return;
  if (evento.target instanceof Element && evento.target.closest(
      "input, textarea, select, [contenteditable]")) return;

  const accion = acciones[evento.key.toLowerCase()];
  if (!accion || !puedeEnviar(accion)) return;
  evento.preventDefault();
  enviarAccion(accion).catch(mostrarError);
});
```

Implementa puedeEnviar según el último estado y si hay una acción en curso. Mientras un POST esté pendiente, deshabilita los botones de acción y rechaza nuevas acciones de teclado. Al terminar, restaura los controles según el nuevo snapshot.

enviarAccion debe usar la cola y POST al ID actual con JSON {accion}. En running, I/E requieren proceso activo. En pausa, C reanuda y P repetida es válida; I/E quedan deshabilitadas. En completed, deshabilita todos los controles y permite una nueva creación.

2. Volver a la misma simulación

Al cargar, lee sessionStorage.getItem("sso.simulacionId"). Si hay ID, consulta por GET, renderiza y arranca un solo ciclo si no terminó. Si no hay ID, muestra el formulario. sessionStorage sirve para recargar esa pestaña; no garantiza recuperar al cerrar la pestaña. Referencia.

Respuesta	Qué mostrar y hacer
422	Entrada o acción inválida. Mostrar el mensaje; no borrar el ID válido.
409	Acción incompatible con el estado actual. Mostrar detail y consultar de nuevo para sincronizar la pantalla.
404	La simulación ya no existe, por ejemplo tras reiniciar el servidor. Borrar el ID guardado, detener consultas y ofrecer iniciar otra.
Sin conexión	Conservar la última vista, deshabilitar acciones y avisar que está desactualizada. Reintentar; no borrar el ID automáticamente.

Reiniciar el servidor borra todas las simulaciones. Guardar cambios en Python con --reload también puede reiniciarlo. Si un 404 aparece después de eso, no significa que el navegador haya perdido el ID.

PASO 08 · INTEGRANTE 3

Comprobar la integración completa

Antes del PR, recorre estos casos desde tu interfaz real. /docs ayuda a comparar la respuesta, pero la entrega debe poder usarse desde la pantalla sin escribir JSON a mano.

Caso	Criterio para aprobar
Arranque y cantidad	API disponible. Cantidad positiva entera. Vacío, 0, negativo o decimal muestran aviso. Doble clic no crea dos simulaciones.
Crear 4 con seed=123 en pruebas	Estado running y reloj 0 al crear. Lote 1, 1 pendiente, proceso 1 activo y espera [2,3]. El seed no tiene que ser un campo de la interfaz normal.
P y C	P congela reloj y tiempos. C conserva lo ejecutado y vuelve a avanzar. En pausa, I/E no se envían.
I antes del reloj 20, con 3 y seed=123	El proceso interrumpido va al final del mismo lote, conserva ejecutado/restante y la pantalla representa la nueva cola.
E durante ejecución	El proceso termina como ERROR, resultado null, restante 0. La pantalla muestra al siguiente activo.
Final de 3 con seed=123, sin E	Reloj 33. En completed: actual y lote null, espera vacía y los tres terminados. Las consultas se detienen.
Teclado y botones	I/E/P/C funcionan. Mantener una tecla no repite acciones. Escribir en el input no dispara controles.
Recarga y reinicio	F5 recupera la misma simulación. Reiniciar el servidor produce aviso de ID inexistente y permite iniciar otra.
Conexión y solicitudes	API caída no borra la vista. Tras recuperarse, se sincroniza. En Red no aparecen GET acumulados ni POST duplicados.
Pantallas pequeñas	Textos legibles, paneles apilados, tablas accesibles y botones utilizables.

Revisión final

```
.\.venv\Scripts\python.exe -m pytest -q
```

Las 56 pruebas de la base deben seguir pasando. pytest no comprueba por sí solo el teclado, el renderizado ni la recuperación en el navegador: registra esas verificaciones en docs/pruebas-frontend.md. Si cambias la forma de cargar scripts, actualiza la prueba de arranque para verificar el comportamiento nuevo.

Abre F12 → Consola y Red. Busca errores de JavaScript, rutas incorrectas o consultas duplicadas. Comparen las cifras con la API entre los tres integrantes. La interfaz básica debe quedar aprobada antes de agregar efectos 3D.

PASO 09 · INTEGRANTE 3

Entregar la interfaz con un PR

Guarda solo los archivos de tu trabajo. Revisa la pantalla con los integrantes 1 y 2. No hace falta modificar el motor ni el servicio para construir esta interfaz.

1. Revisar, guardar y publicar

```
git branch --show-current
git status
git diff --stat
.\.venv\Scripts\python.exe -m pytest -q
git add frontend README.md docs/pruebas-frontend.md
git diff --cached
git commit -m "feat: conectar interfaz con API de simulacion"
git push -u origin feature/i3-interfaz
```

Si actualizaste una prueba o el contrato de forma acordada, agrega esos archivos expresamente. .venv, cachés y archivos ajenos a tu trabajo quedan fuera. Después de la primera subida puedes usar git push.

2. Describir y abrir el PR desde PowerShell

```
$prBodyPath = Join-Path $env:TEMP "pr-frontend-i3.md"
@'
Conecta la pantalla con la API ya integrada.
Incluye inicio, consulta, tablas, teclas I/E/P/C y recuperación.

Verificación:
- Pruebas de Python aprobadas.
- Casos del navegador registrados en docs/pruebas-frontend.md.
- Revisado con los integrantes 1 y 2.

La mejora 3D se propone después de esta integración.
'@ | Set-Content -Encoding utf8 $prBodyPath
gh pr create --base main --head feature/i3-interfaz `
  --title "Integrar interfaz de simulacion" `
  --body-file $prBodyPath
gh pr view --web
```

Usa la descripción solo cuando esas comprobaciones realmente estén hechas. Comparte el enlace y solicita revisión. Si te piden ajustes, modifica, prueba, crea otro commit y sube a la misma rama: se actualiza el mismo PR.

3. Después de integrar

```
git switch main
git pull --ff-only origin main
```

Prueben juntos la versión de main: crear, consultar, I/E/P/C, final y recarga. Si la interfaz cumple todo, ya pueden abrir una nueva rama para la mejora opcional de las siguientes páginas.

MEJORA OPCIONAL 10 · INTEGRANTE 3

Al final: ideas con Three.js

Sugerencia opcional del equipo. Empezar únicamente después de integrar y aprobar motor, API y frontend funcional. La animación debe explicar el estado recibido, conservando las tablas y controles.

Idea recomendada: estación de procesamiento



Una cámara fija en vista isométrica, una zona de espera a la izquierda, una CPU al centro y una bandeja de terminados a la derecha. Cada trabajo es un cubo pequeño con su ID. Es suficiente para explicar los lotes sin construir un escenario complejo.

Dato o acción	Cómo representarlo
ready / en_espera	Cubos azules en el mismo orden del JSON. Mostrar ID y tiempo restante con etiquetas HTML.
running / en_ejecucion	Cubo turquesa en la CPU. Un aro o barra muestra ejecutado / TME usando los valores recibidos.
I: interrupción	Mover el cubo de la CPU al final de la fila; después representar el siguiente activo.
E: error	Cubo rojo en terminados y etiqueta ERROR. Sin depender solo del color.
P / C	En pausa, congelar los movimientos y mostrar “Pausada”. Al continuar, actualizar a partir de la respuesta.
completed	CPU vacía, escena estable y resumen final. Mostrar reloj y resultados en HTML.

Otras ideas, si queda tiempo

Bandejas de lote: iluminar la bandeja del lote actual; representar lotes pendientes con contenedores genéricos y su cantidad.
Circuito de flujo: una pista sencilla entre espera, CPU y terminados, con un pequeño destello al cambiar de proceso.

El snapshot no incluye los trabajos de los lotes futuros. No inventes sus IDs, operaciones o tiempos. Tampoco garantiza capturar todas las transiciones entre consultas: anima el cambio visible hacia el estado nuevo, sin fabricar eventos.

MEJORA OPCIONAL 11 · INTEGRANTE 3

Agregar Three.js sin cambiar la API

Mantén la mejora en un módulo independiente y en un PR posterior. Puedes empezar con cubos, una cámara fija y luces sencillas; no necesitas modelos 3D externos.

1. Preparar la rama y los archivos opcionales

```
git switch main
git pull --ff-only origin main
git switch -c feature/i3-visual-threejs
```

Crea frontend/scene.js para la escena. En index.html agrega un canvas y un botón “Ver animación”. En styles.css define su tamaño. app.js conserva las solicitudes y envía el último snapshot al módulo 3D.

2. Cargarlo solo cuando se solicite

```
// En scene.js, un módulo JavaScript:
import * as THREE from
  "https://cdn.jsdelivr.net/npm/three@0.180.0/build/three.module.js";

// Implementar y exportar:
// initScene(canvas): crea escena, cámara, luces y renderer.
// updateScene(snapshot): actualiza objetos desde el JSON.
// disposeScene(): libera recursos al cerrar la visualización.
```

0.180.0 es una versión fija de ejemplo, verificada; no se presenta como la más reciente. La carga por CDN sirve para empezar sin cambiar el servidor. Si se usan addons, mantener la misma versión y configurar imports conforme a la guía oficial de instalación.

```
// En el controlador del botón, dentro de app.js:
try {
  const modulo = await import("/scene.js");
  modulo.initScene(document.querySelector("#escena"));
  if (ultimoSnapshot) modulo.updateScene(ultimoSnapshot);
  vista3D = modulo;
} catch (error) {
  mostrarAviso("La vista 3D no pudo cargarse");
}
```

Declara ultimoSnapshot y vista3D en el controlador. Evita inicializar dos veces al repetir clic. Cada renderizar(snapshot) guarda ultimoSnapshot, actualiza primero el HTML y después llama updateScene si la escena está lista. Captura también errores del módulo para conservar la interfaz funcional.

3. Para una demo sin internet

El CDN necesita conexión. Si la demo debe funcionar sin internet, guarda localmente los módulos de esa misma versión y su licencia en frontend/vendor/three/. Incluye three.module.js y three.core.js, más las dependencias de los addons elegidos. Cambia el import al archivo local y prueba con internet desconectado.

MEJORA OPCIONAL 12 · INTEGRANTE 3

Reglas de la visualización 3D

La animación es una representación opcional. El mismo snapshot alimenta HTML y Three.js: no hace solicitudes adicionales ni toma decisiones sobre la simulación.

Piezas mínimas de scene.js

Pieza	Función
Scene y cámara	Contener los objetos y encuadrar espera, CPU y terminados. Una cámara fija facilita empezar.
WebGLRenderer y canvas	Dibujar dentro de la zona 3D. Actualizar el tamaño del renderer y la cámara al cambiar el contenedor.
BoxGeometry y materiales	Construir los cubos. Identificarlos por ID mediante un Map para mover el mismo objeto, no duplicarlo.
Luces sencillas	Dar volumen sin efectos costosos. Evitar sombras, texturas grandes y postprocesado al principio.
Ciclo de render	Actualizar animaciones visuales hacia posiciones objetivo. No llamar tick ni avanzar reloj en cada frame.

Cómo mantenerla fiel al estado

updateScene(snapshot) compara objetos por ID, coloca el actual en la CPU, acomoda la cola en el orden recibido y añade los terminados. El progreso sale de ejecutado/tme. Conserva la cifra oficial del reloj en HTML. Si se pierde conexión, detén el movimiento y muestra el estado desactualizado.

Congela las transiciones cuando snapshot.estado sea paused y al finalizar deja una escena estable. Mantén las etiquetas de ID, ERROR y tiempos en HTML para que sean legibles. Un botón permite ocultar la animación; la simulación debe poder usarse completa sin ella.

Comprobar antes del segundo PR

Rendimiento: empieza con pocos cubos; reutiliza geometrías y materiales. Muestra solo el lote actual y limita la vista 3D de terminados, manteniendo todos en la tabla.
Tamaño: adapta canvas y cámara al contenedor; limita la densidad de píxeles en equipos modestos.
Cierre: detén el ciclo y libera renderer, geometrías y materiales con dispose al desmontar la escena.
Compatibilidad: prueba sin soporte 3D o con carga fallida. La interfaz básica debe seguir funcionando.

Documenta que el 3D es opcional, registra sus pruebas y abre otro PR hacia main. Usa el mismo flujo de commit/push/revisión de la interfaz, cambiando el nombre de rama y la descripción. No agregues nuevas rutas ni cambios al motor para dibujar los cubos.

Referencias clicables

Código de main y contrato del proyecto revisados el 12/10/2026. Para implementación: fetch, teclas repetidas, sessionStorage y PR desde terminal.

Three.js: instalación, fundamentos, tamaño adaptable, liberar recursos y versión r180 del ejemplo. Las ideas visuales de esta guía son propuestas para este proyecto.